

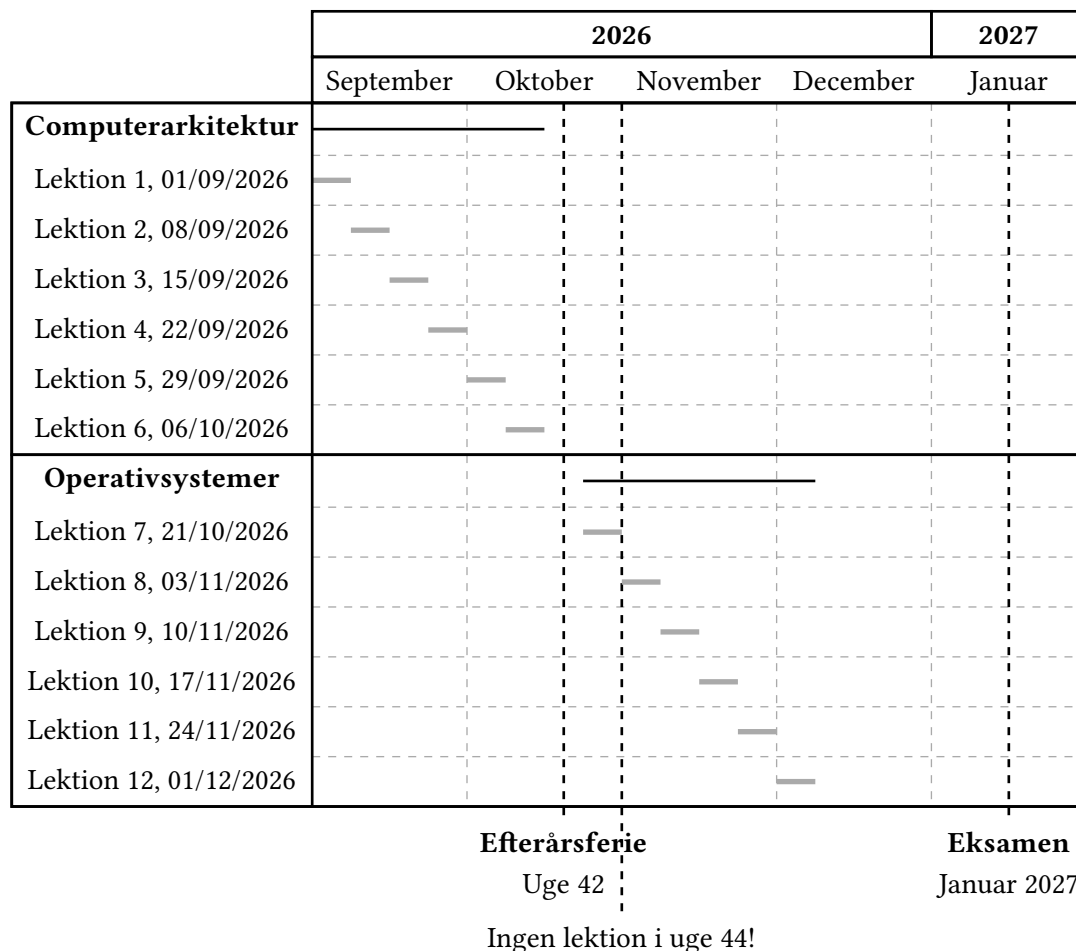
COMPUTERARKITEKTUR OG OPERATIVSYSTEMER 2026

T540042102

Emil Lykke Diget

22 September 2026

Velkommen til kurset “Computerarkitektur og Operativsystemer”. I dette dokument finder du en oversigt over de tolv lektioner, deres litteratur og øvelsesopgaver. Litteraturlisten kan findes til sidst i dokumentet. På Figur 1 ses en oversigt over lektionerne i løbet af semesteret illustreret i et Gantt-diagram. Kurset er delt op i to distinkte dele; *Computerarkitektur* og *Operativsystemer*. Eksamen ligger i løbet af januar 2027; der ligger lidt information om eksamen i slutningen af dette dokument.



Figur 1: Gantt-diagram for lektionerne i kurset. Bemærk venligst, at der ikke er skemalagte lektioner i uge 42 og 44.

Indhold

Lektion 1: Kombinatoriske Logiske Kredse	3
Lektion 2: En Sempel Computer	4
Lektion 3: Hukommelse	5
Lektion 4: Mikroarkitektur	6
Lektion 5: Micro-assembly og IJVM	8
Lektion 6: Optimering af Mikroarkitekturdesign	10
Lektion 7: Introduktion til Operativsystemer	11
Lektion 8: Tråde og CPU-Schedulering	12
Lektion 9: Proces-synkronisering	13
Lektion 10: Deadlocks	17
Lektion 11: Memory Management	18
Lektion 12: Virtuel Hukommelse	19
Eksamen	20
Litteraturliste	21

Lektion 1: Kombinatoriske Logiske Kredse

Denne lektion introducerer forskellige talsystemer, herunder det binære, octal, hexadecimal og titalssystemet, og det binære talsystem som grundlag for boolsk algebra og digitale kredsløb.

Læsestof

Læs følgende i [1]:

- Læs Appendix A *Binary Numbers* s. 669–678
- Afsnit 3.1–3.2 i Kapitel 3 *The Digital Logic Level* s. 147–168

Opgave

1. Opskriv og reducer udtrykket, der kommer fra følgende sandhedstabel:

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

2. Reducér følgende udtryk:

$$X = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

3. Løs opgave 6 s. 679 i Appendix A i [1].
4. Løs opgave 7 s. 679 i Appendix A i [1].
5. Løs opgave 14 s. 680 i Appendix A i [1].
6. Betragt kredsløbet i opgave 8 s. 238 i [1] og opskriv en sandhedstabel for denne.
7. Implementér en 8-bit ALU i en digital-logik simulator som f.eks. *Digital* <https://github.com/hneemann/Digital>.

Lektion 2: En Simpel Computer

Denne lektion introducerer Von Neuman-arkitekturen, der er en tidlig beskrivelse af computeren med opdeling af CPU, hukommelse og IO. Moderne computere har fællestræk med Von Neuman-arkitekturen.

Læsestof

- Læs afsnit 1.2 s.13–28 i [1]
- Læs afsnit 2.1 s. 55–73 i [1]
- Læs afsnit 2.1–2.4 s. 18–43 i Kapitel 2 i [2] (findes på Itslearning)

Opgave

1. Find databladet for CPUen i *jeres* PC. Hvordan adskiller dens arkitektur sig fra fra den klassiske Von Neuman-arkitektur?
2. Implementér to forskellige kontrolenheder med `load`-operationen i *Digital*
 - Horisontal instruktionsdesign
 - Vertikal instruktionsdesign

Lektion 3: Hukommelse

Denne lektion drejer sig om, hvad hukommelse (memory) er, og hvordan man forbinder ekstern hukommelse til en CPU.

Læsestof

- Læs hhv. side 23-33 og 331 i [3]
- Læs følgende i [1]:
 - Afsnit 2.2 *Primary Memory* s. 73–85
 - Afsnit 3.3 *Memory* s. 169–185
 - Afsnit 3.7 *Interfacing* s. 232–235

Opgave

Opgave 3.1

En mikroprocessor har en 14 bit adresse-bus. Tegn et memory-map af adresseområdet, hvor grænserne angives med hexadecimale og binære værdier. En 4K ROM-kreds er placeret i den nederste del af adresseområdet, startende med den laveste adresse. En 2K RAM er placeret i den øverste del af adresseområdet, sluttende med den højeste adresse. Indtegn hukommelseskredsene i adresseområdet, hvor grænserne angives med hexadecimale og binære værdier.

Opgave 3.2

Betrakt et system bestående af en mikroprocessor med en 20 bits adresse-bus og en 8 bit databus, 2 stk. $2K \times 8$ RAM-kredse og 1 stk. $8K \times 8$ ROM-kreds. Den første RAM-kreds skal placeres i hukommelsesområdet, så den starter på den laveste adresse. Den anden RAM-kreds skal placeres, så der er et ubrugt område (not used) mellem den første RAM-kreds og starten af den anden kreds på $4K \times 8$. ROM-kredsen skal placeres i den øverste del af adresseområdet, sluttende med den højeste adresse. Indtegn hukommelseskredsene i adresseområdet, hvor grænserne angives med hexadecimale og binære værdier.

Opgave 3.3

Design den nødvendige dekodningslogik til opgave 3.1 med diskrete logiske kredse.

Opgave 3.4

Design den nødvendige dekodningslogik til opgave 3.2 med diskrete logiske kredse.

Lektion 4: Mikroarkitektur

Denne lektion drejer sig om mikroarkitektur-niveauet. Vi kigger nærmere på Mic-1-CPU'en og dens data-path, hukommelse, mikroinstruktioner og instruktionssæt.

Læsestof

Læs Kapitel 4 *The Microarchitecture Level* s. 243–267 indtil afsnit 4.3 i [1].

Opgave

Opgave 4.1

Bestem ud fra nedenstående mikroinstruktioner hvilken værdi, der gemmes i `OPC`-registeret. Antag at programudførelsen starter i adresse `0x000`.

Microstore- adresse	Mikroinstruktion					
	Addr	JAM	ALU	C	Mem	B
0x000	000000001	000	00010100	100000000	000	0101
0x001	000000010	000	00111100	010000000	000	0011
0x002	000000010	000	00000000	000000000	000	0000
0x003	000000000	000	00000000	000000000	000	0000

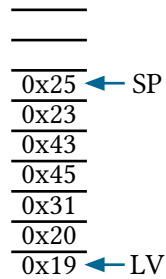
Opgave 4.2

Bestem funktionen af nedenstående mikroinstruktioner. Antag at programudførelsen starter i adresse `0x000`.

Microstore- adresse	Mikroinstruktion					
	Addr	JAM	ALU	C	Mem	B
0x000	000000001	000	00110111	000000001	010	0100
0x001	000000010	000	00000000	000000000	000	0000
0x002	000000011	000	00010100	110000000	000	0000
0x003	000000011	000	00000000	000000000	000	0000

Opgave 4.3

Bestem stakkens udssende samt hvor SP og LV peger efter udførelsen af nedenstående kode. Programudførelsen starter i adresse 0x010 og slutter i adresse 0x00F.



Microstore- adresse	Mikroinstruktion					
	Addr	JAM	ALU	C	Mem	B
0x00E	000000011	000	00010100	110000000	000	0000
0x00F	000001111	000	00000000	000000000	000	0000
0x010	000010001	000	00110101	010000000	000	0101
0x011	000010010	000	00110101	000000001	010	1000
0x012	000010011	000	00110101	000001001	000	0100
0x013	000010101	000	00010100	001000000	100	0000
0x014	000010010	000	00111100	000000001	010	0101
0x015	000001111	000	00000000	000000000	000	0000
0x016	000000000	000	00000000	000000000	000	0000

Opgave 4.4

Skriv et mikroinstruktionsprogram, der lægger de to øverste værdier af stakken sammen og gemmer resultatet øverst på stakken.

Hvor lang tid tager det at afvikle programmet med en clock-frekvens på 100 MHz?

Opgave 4.5

Betragt nedenstående pseudokode.

```

1 while (i < 10) {
2     i++;
3 }
4 k = i;
```

Variabel `i` er i lokal-variable 1 og variable `k` er lokal variabel 2. Antag at variablen 0 og at `H`-registeret indeholder værdien 10.

Skriv hhv. et mikroprogram og et IJVM-program, der implementerer pseudokoden.

Lektion 5: Micro-assembly og IJVM

Denne lektion drejer sig om programmering af mikroarkitekturen. Vi kigger igen på Mic-1 og Micro Assembly Language.

Læsestof

Læs Kapitel 4 *The Microarchitecture Level* s. 267–282 indtil afsnit 4.4 i [1].

Opgave

Opgave 5.1

1. Start simulatoren efter installation
2. Klik på “Mic-1 MMV Help” i “About”-menuen
3. Tryk på krydset ud fra “Getting started” i menuen til venstre
4. Gennemgå de fire eksempler.

Opgave 5.2

I mikroprogrammet til Mic-1 kan man se, at `MDR` kopieres til `H` i linien `if_icmpeq3`. Nogle få linier senere, trækkes det fra `TOS` for at tjekke, om de er ens. Hvorfor kan man ikke bare skrive sådan her:

```
1 if_icmpeq3 Z = TOS - MDR; rd
```

→ if_icmpeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
if_icmpeq2	MAR = SP = SP - 1	Set MAR to read in new top-of-stack
if_icmpeq3	H = MDR; rd	Copy second stack word to H
if_icmpeq4	OPC = TOS	Save TOS in OPC temporarily
if_icmpeq5	TOS = MDR	Put new top of stack in TOS
→ if_icmpeq6	Z = OPC - H; if (Z) goto T; else goto F	If top 2 words are equal, goto T, else goto F

Opgave 5.3

Tilføj JVM-instruktionen `POP2` til instruktionssættet i mikrokode (`mic1ijvm.mal`) og skriv herefter et IJVM-program, der benytter denne instruktion og test resultatet i Mic-1-simulatoren.

0x58	POP2
Beskrivelse:	Instruktionen popper to words fra stakken

Bemærk. Denne instruktion har koden `0x58`, altså skal der tilføjes et nyt label til `mic1ijvm.mal` og implementere funktionen i `ijvm.conf`.

Opgave 5.4

Tilføj JVM-instruktionen `IF_ICMPLT` til instruktionssættet i mikrokoden (`mic1ijvm.mal`) og skriv herefter et IJVM-program, der benytter denne instruktion og test resultatet i Mic-1-simulatoren.

0xA1	IF_ICMPLT <label>
Beskrivelse:	Instruktionen popper to words fra stakken og udfører et jump, hvis den næstøverste værdi på stakken er mindre end den øverste værdi.

Lektion 6: Optimering af Mikroarkitekturdesign

Denne lektion drejer sig om optimering af mikroarkitekturen. Vi kigger på problemer ved Mic-1s struktur, og hvad man kan gøre ved det.

Optimering af arkitekturen leder til Mic-2 og Mic-3.

Læsestof

Læs følgende sider i [1];

- om pipelining i Sektion 2.1.5 s. 65–67 indtil *Superscalar Architectures*
- om cache i Sektion 2.2.5 s. 82–85
- i kapitel 4 *The Microarchitecture Level* s. 283–300 indtil afsnit 4.4.5
- i kapitel 4 *The Microarchitecture Level* s. 303–310 indtil afsnit 4.5.2

Opgave

1. Oversæt følgende instruktioner fra Mic-2 til den pipelinede Mic-3 og sammenlign antal cyklusser.
 - IADD
 - ISTORE
 - ILOAD
 - POP
 - INVOKEVIRTUAL
2. Løs opgave 4.32 i [1] s. 341. Benyt dit foretrukne sprog, f.eks. Python, C++, etc.

Lektion 7: Introduktion til Operativsystemer

Vi er nået til den anden halvdel af kurset, der drejer sig om operativsystemer. Denne lektion er en introduktion til operativsystemer, dets funktioner og dets struktur. Vi tager også et kig på processor.

Læsestof

Læs følgende i [4]:

- Kapitel 1 *Introduction*, afsnit 1.1, 1.2, 1.4, 1.9
- Kapitel 2 *Operating-System Structures*, afsnit 2.1–2.8
- Kapitel 3 *Processes*, afsnit 3.1–3.7

Opgave

Løs følgende opgaver i [4]

1. Løs opgave 3.9
2. Løs opgave 3.10
3. Løs opgave 3.12

Snak først koden i bogen igennem med hinanden og bliv enige om, hvad den burde gøre. Først derefter skal I skrive, kompilere og køre programmet på computeren.

Fik I ret? Gør koden det, I forventede?

Bemærk. Koden i bogen mangler `#include <sys/wait.h>` for at kunne bruge `wait`-funktionen.

Bemærk. C kan kompileres på eksempelvis Linux ved brug af kompilatoren `gcc` ved at køre følgende kommando i terminalen:

```
1 $ gcc ex3_09.c -o ex3_09.o
```

hvor `ex3_09.c` indeholder koden til opgave 3.9 og `ex3_09.o` er det resulterende eksekverbare program. Det kan eksekveres ved:

```
1 $ ./ex3_09.o
```

Lektion 8: Tråde og CPU-Schedulering

Denne lektion handler om tråde, der muliggør parallelisme i ens program. Med parallelisme følger dog udfordringen, at CPUen kun kan udføre én opgave af gangen, så den er nødt til at skifte imellem processer og tråde. Dette problem forsøges løses med CPU-schedulering.

Læsestof

Læs følgende i [4]

- Kapitel 4 *Threads & Concurrency*, afsnit 4.1–4.3
- Kapitel 5 *CPU Scheduling*, afsnit 5.1–5.6, 5.8

Opgave

1. Løs opgave 4.9 i [4]
2. Løs opgave 4.12 i [4]
Diskuter først, og prøv at lave en implementering af koden og se, om I svarede rigtigt.
3. Løs opgave 4.13 i [4]
Diskuter først, og prøv at lave en implementering af koden og se, om I svarede rigtigt.
4. Løs opgave 5.15 i [4]

Lektion 9: Proces-synkronisering

Med flere processer kørende på computeren samtidigt, der muligvis skal bruge de samme ressourcer, kan der opstå problemer, f.eks. hvis to processer ønsker adgang til den samme ressource på samme tid. I denne lektion skal vi derfor kigge på, hvordan man synkroniserer processer vha. værktøjer som mutex-låse, semaforer, monitorer, etc.

Læsestof

Læs følgende i [4]

- Kapitel 6 *Process Synchronization*, afsnit 6.1–6.8
- Kapitel 7 *Synchronization Examples*, afsnit 7.1, 7.3

Opgave

1. Løs opgave 6.7 i [4]
2. Implementer de tre synkroniseringsproblemer præsenteret i lektionen i C vha. POSIX-semaforer og delt hukommelse. Brug følgende skabelon for at oprette delt hukommelse og semaforer, her er det til bounded-buffer-problemet.

producer.c :

```
1  #include <pthread.h>
2  #include <semaphore.h>
3
4  #include <stdio.h>
5  #include <fcntl.h>
6  #include <sys/shm.h>
7  #include <sys/stat.h>
8  #include <sys/mman.h>
9  #include <unistd.h>
10
11 #define NUMINTS (5)
12 #define FILESIZE (NUMINTS * sizeof(int))
13
14 int main()
15 {
16     // name of shared memory object
17     const char* name = "BUFFER";
18     // shared memory file descriptor
19     int shm_fd;
20     // pointer to shared memory object
21     int* buffer_ptr;
22     // create object
```

```
23  shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
24  // configure size
25  ftruncate(shm_fd, FILESIZE);
26  // memory map of the shared memory object
27  buffer_ptr = mmap(0, FILESIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
28
29  // Create semaphores
30  printf("creating semaphores\n");
31  sem_unlink("SEM_Mutex");
32  sem_t *mutex;
33  mutex = sem_open("SEM_Mutex", O_CREAT, 0666, 1);
34  sem_unlink("SEM_EMPTY");
35  sem_t *empty;
36  empty = sem_open("SEM_EMPTY", O_CREAT, 0666, NUMINTS);
37  sem_unlink("SEM_FULL");
38  sem_t *full;
39  full = sem_open("SEM_FULL", O_CREAT, 0666, 0);
40
41  int next_produced = 0;
42  int* tos;
43
44  for (;;) // while true loop
45  {
46      // produce stuff,
47      // but make sure to use semaphores when accessing the buffer
48      // get top of stack
49      sem_getvalue(empty, tos);
50
51      sleep(1);
52  }
53
54  // Free the mapped memory
55  if (munmap(buffer_ptr, FILESIZE) == -1) {
56      perror("Error un-mapping the file");
57      /* Decide here whether to close(fd) and exit() or not. Depends... */
58  }
59  // close file
60  close(shm_fd);
61
62  return 0;
63 }
```

`consumer.c :`

```
1  #include <pthread.h>
2  #include <semaphore.h>
3
4  #include <stdio.h>
5  #include <fcntl.h>
6  #include <sys/shm.h>
7  #include <sys/stat.h>
8  #include <sys/mman.h>
9  #include <unistd.h>
10
11 #define NUMINTS (5)
12 #define FILESIZE (NUMINTS * sizeof(int))
13
14 int main()
15 {
16     // name of shared memory object
17     const char* name = "BUFFER";
18     // shared memory file descriptor
19     int shm_fd;
20     // pointer to shared memory object
21     int* buffer_ptr;
22     // create object
23     shm_fd = shm_open(name, O_RDONLY, 0666);
24     // memory map of the shared memory object
25     buffer_ptr = mmap(0, FILESIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
26     // Get semaphores
27     sem_t *mutex;
28     mutex = sem_open("SEM_MUTEX", O_CREAT, 0666, 1);
29     sem_t *empty;
30     empty = sem_open("SEM_EMPTY", O_CREAT, 0666, NUMINTS);
31     sem_t *full;
32     full = sem_open("SEM_FULL", O_CREAT, 0666, 0);
33
34     // Do consumer stuff
35     int next_consumed = 0;
36     int* tos;
37
38     for (;;)
39     {
40         // get stop of stack value
41         sem_getvalue(empty, tos);
42         // access buffer and consume, make sure to use semaphores
43
44         sleep(1);
45     }
```

```
46
47 // Free the mmaped memory
48 if (munmap(buffer_ptr, FILESIZE) == -1) {
49     perror("Error un-mmapping the file");
50     /* Decide here whether to close(fd) and exit() or not. Depends... */
51 }
52
53 // close file
54 close(shm_fd);
55
56 return 0;
57 }
```

Lektion 10: Deadlocks

Hvis flere tråde bruger de samme begrænsede ressourcer, kan man risikere, at de ender i et `wait`-state. Hvis ressourcen holdes af en tråd i `wait`-state, kan den oprindelige tråd vente for evigt. Dette kaldes for *deadlock*, og er det, vi skal beskæftige os med i dag.

Læsestof

Læs følgende i [4]

- Kapitel 8 *Deadlocks*, afsnit 8.1–8.8

Opgave

1. Løs opgave 8.12 i [4]
2. Løs opgave 8.14 i [4]
3. Løs opgave 8.15 i [4]
4. Løs opgave 8.16 i [4]

Lektion 11: Memory Management

Tilbage i Lektion 3 kiggede vi på hukommelse ud fra et hardware-synspunkt – I dag skal vi kigge på hukommelse fra operativsystemets synspunkt.

Læsestof

Læs følgende i [4]

- Kapitel 8 “*Memory Management*”, afsnit 9.1–9.6

Opgave

1. Løs opgave 9.16 i [4]
2. Løs opgave 9.17 i [4]
3. Løs opgave 9.18 i [4]

Lektion 12: Virtuel Hukommelse

Ofte vil man rende ind i det problem, at man har brug for mere hukommelse, end man fysisk har. *Virtuel hukommelse* er en teknik til at eksekvere programmer, der ikke kan være fuldstændigt i hukommelsen. Her taler man om *logisk* hukommelse, som en programmør kan tilgå, og så den faktiske fysiske hukommelse i hardware.

Læsestof

Læs følgende i [4]:

- Kapitel 9 “*Virtual Memory*”, afsnit 10.1–10.6

Opgave

1. Løs opgave 10.17 i [4]
2. Løs opgave 10.19 i [4]

Eksamen

Dette kursus afsluttes af en individuel mundtlig eksamen. I slutningen af kurset, vil jeg offentliggøre fire eksamensemner. På dagen for eksamen trækker hver studerende et emne og giver en 8 min. præsentation. Efter den studerendes præsentation er der 8 min. spørgsmål til både præsentationen og resten af pensum.

Præstationen bedømmes efter 7-trinsskalaen, som I kender det fra andre fag.

Eksamensrækkefølgen vil blive offentliggjort i god tid inden eksamen.

Litteraturliste

- [1] A. S. Tanenbaum, T. Austin, og B. Chandavarkar, *Structured computer organization*, 6. edition. International edition. Boston, Mass: Pearson, 2013.
- [2] M. R. Zargham, *Computer architecture: single and parallel systems*. Prentice-Hall, Inc., 1996.
- [3] Atmel, "ATmega128L Datasheet". 2011.
- [4] A. Silberschatz, J. L. Peterson, og P. B. Galvin, *Operating system concepts*, 10. edition. Global Edition. John Wiley & Sons Inc., 2019.